# SOLAR RADIATION PREDICTION BY USING MACHINE LEARNING

*A Long Term Internship Project report submitted to*

**ADIKAVI NANNAYA UNIVERSITY-Rajamahendravaram**

*Submitted in partial fulfillment of the requirements*

*for the award of degree*

**BACHELOR OF DEGREE**

**In**

**BACHELOR OF COMPUTER APPLICATIONS**

**(DATA SCIENCE)**

**Submitted by**


**MADADHA RIBKA**

**230217402025**

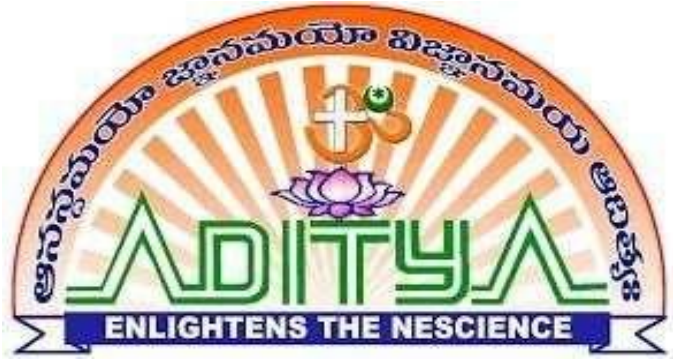

*Under the esteemed Guidance of*

**Mr. K. PAVAN KUMAR**

**TANVIKA SOLUTIONS PVT. LTD**

*Under the Internal Guidance of*

**Mr. D.RAJA (MSC)**




## Department of BCA-DATA SCIENCE


# ADITYA DEGREE COLLEGE, KAKINADA

**Permanent Affiliation to AKNU, Rajamahendravaram**

**Accredited by NAAC with A++ Grade**

**An ISO 9001:2015 Certified Institution**

**Lakshmi Narayana nagar, Kakinada(Dist)- 533003**

**Ph : 0884 2376665, 9704376667 | e-mail adckkd@aditya.ac.in**

**Website :** www.aditya.ac.in/degreecolleges


(2025 – 2026)

# ADITYA DEGREE COLLEGE

**Permanent Affiliation to AKNU, Rajamahendravaram**

**Accredited by NAAC with A++ Grade**

**An ISO 9001:2015 Certified Institution**

**Lakshmi Narayana nagar, Kakinada (Dist)- 533003**

**Ph : 0884 2376665, 9704376667 | e-mail adckkd@aditya.ac.in**

**Website :** www.aditya.ac.in/degreecolleges

## Department of BCA-DATA SCIENCE


# Certificate

**This is to certify that the project work entitled *"SOLAR RADIATION PREDICTION BY USING MACHINE LEARNING"* is a Bonafide record work done by** MADADHA RIBKA & 230217402025.

**In the Department of BCA - Data SCIENCE, ADITYA DEGREE COLLEGE, is affiliated to AKNU - Rajamahendravaram in partialfulfillment of the requirements for the award of Bachelor of Degree in BCA - Data science during 2023-2026. This work has been carried out under my guidance and supervision.**

**The results embodied in this Project report have not been submitted in any University or Organization for the award of any degree.**

**INTERNSHIP GUIDE:**
Mr. K. PAVAN KUMAR

**INTERNAL GUIDE:**
Mr. D.RAJA (MSC)

**HEAD OF THE DEPARTMENT**
Mr. V. S. N. KUMAR

**VICE PRINCIPAL:**
Mr. C. SATYA NARAYANA

**External Examiner**

# ACKNOWLEDGEMENTS

I feel it is my duty and honor to acknowledge all those who have extended their guidance and warm support in completing my project work.

Firstly, it is my privilege to thank **Sri N. SESHA REDDY**, Chairman, Aditya Group of Educational Institutions for providing state of the art facilities, experienced and talented faculty members.

I earnestly convey my thanks to, **Smt. N, SUGUNA REDDY**, Secretary of Aditya Educations Institutions for making me use all the technical facilities in the college.

I thank to **Sri B.E.V.L. NAIDU**, Academic Director and Principal of Aditya Educational Institutions for providing wonderful Academic curriculum and enhancement programs for us.

I am grateful to **Mr. C. SATYA NARAYANA,** Vice Principal of Aditya Degree College, Kakinada for continuous support and encouragement in my endeavor.

I Also thank **Mr. V.S.N. KUMAR**., Head of the Department for continuous support for completing my project.

I Also thank **Mr. N. KIRAN KUMAR**., Internal Guide of the Project for continuous support for completing my project.

I Also thanking **Mr. K. PAVAN KUMAR**, guide of our project and head of the organization **SRI. B. SUNIL KUMAR REDDY** for the support render by them and express my deep sense of gratitude to her under her guidance I could make a through and complete copy of my project work.

We are also thankful to all the staff members of BCA-DS department who have co-operated in making our project a success. We would like to thank all our parents and friends who extendedtheir help, encouragement and moral support either directly or indirectlyin our project work.

Thanks for Your Valuable Guidance and kind support.

# TANVIKA SOLUTIONS PVT LTD

# CERTIFICATE

## OF INTERNSHIP

PROUDLY PRESENTED TO:

***MADADHA RIBKA***

***REG NO:230217402025***

**OF ADITYA DEGREE COLLEGE, KAKINADA**

Underwent internship in

**Machine Learning using Python**

*From* _____ *to* _____

The overall performance of intern during internship is found to be satisfactory

Head Of Organization

Project Coordinator

# DECLARATION

We here by declare that project report entitled **"SOLAR RADIATION PREDICTION BY USING MACHINE LEARNING"** is a genuine project work carried out by us, in **BCA(Data Science)**degree course of **ADIKAVI NANNAYA UNIVERSITY-Rajamahendravaram** and has not been submitted to any other courses or University for award of any degree by us.

**Signature of the Students**

**MADADHA RIBKA**

**230217402025**

# INDEX

# LIST OF FIGURES

# CHAPTER-1
# INTRODUCTION

# INTRODUCTION

Energy is one of the most important resources for the development of any country. Modern industries, transportation systems, communication networks, hospitals, educational institutions, and households all depend heavily on electricity. Traditionally, electricity has been generated using fossil fuels such as coal, oil, and natural gas. However, these resources are limited in quantity and contribute significantly to environmental pollution and global warming. Due to these challenges, there is a growing demand for renewable and sustainable energy sources.

Solar energy is one of the most abundant and environmentally friendly renewable energy sources available on Earth. The sun continuously emits a massive amount of energy in the form of electromagnetic radiation. This radiation, when captured using solar panels, can be converted into electrical energy. Unlike fossil fuels, solar energy is clean, sustainable, and available almost everywhere. Because of these advantages, many countries are investing heavily in solar power plants and rooftop solar systems.

Solar radiation refers to the amount of solar energy received per unit area at a particular location and time. It is usually measured in watts per square meter ($W/m^2$). The amount of solar radiation reaching the Earth's surface depends on several atmospheric and environmental factors such as temperature, humidity, cloud cover, wind speed, air pressure, and geographical location. These factors vary continuously, making solar radiation highly dynamic and difficult to predict accurately.

Accurate prediction of solar radiation is extremely important for efficient solar power generation. Solar power plants rely on radiation forecasts to estimate the amount of electricity that can be generated. Energy grid operators need reliable predictions to balance supply and demand. Inaccurate forecasts may lead to power shortages or wastage of generated electricity.

Therefore, improving solar radiation prediction models has become an active area of research. Traditional methods of solar radiation forecasting include physical models and statistical regression techniques. Physical models use atmospheric equations and satellite data, but they are complex and computationally expensive. Statistical models, such as linear regression, are simpler but often fail to capture the nonlinear relationships between weather parameters and solar radiation.

In recent years, Machine Learning (ML) has emerged as a powerful tool for solving complex prediction problems. ML algorithms can automatically learn patterns from historical data without being explicitly programmed. They are capable of handling nonlinear

relationships and large datasets efficiently. By training ML models on historical weather and radiation data, it is possible to build intelligent systems that provide highly accurate predictions. This project focuses on developing a Solar Radiation Prediction System using Ensemble Machine Learning techniques. Ensemble methods combine multiple machine learning models to produce better performance than individual models. By integrating algorithms such as Random Forest, Gradient Boosting, and XGBoost, the proposed system aims to improve prediction accuracy, reduce overfitting, and enhance reliability. The developed system will help in: Optimizing solar power plant operations Supporting renewable energy planning Improving grid stability Assisting researchers and policymakers

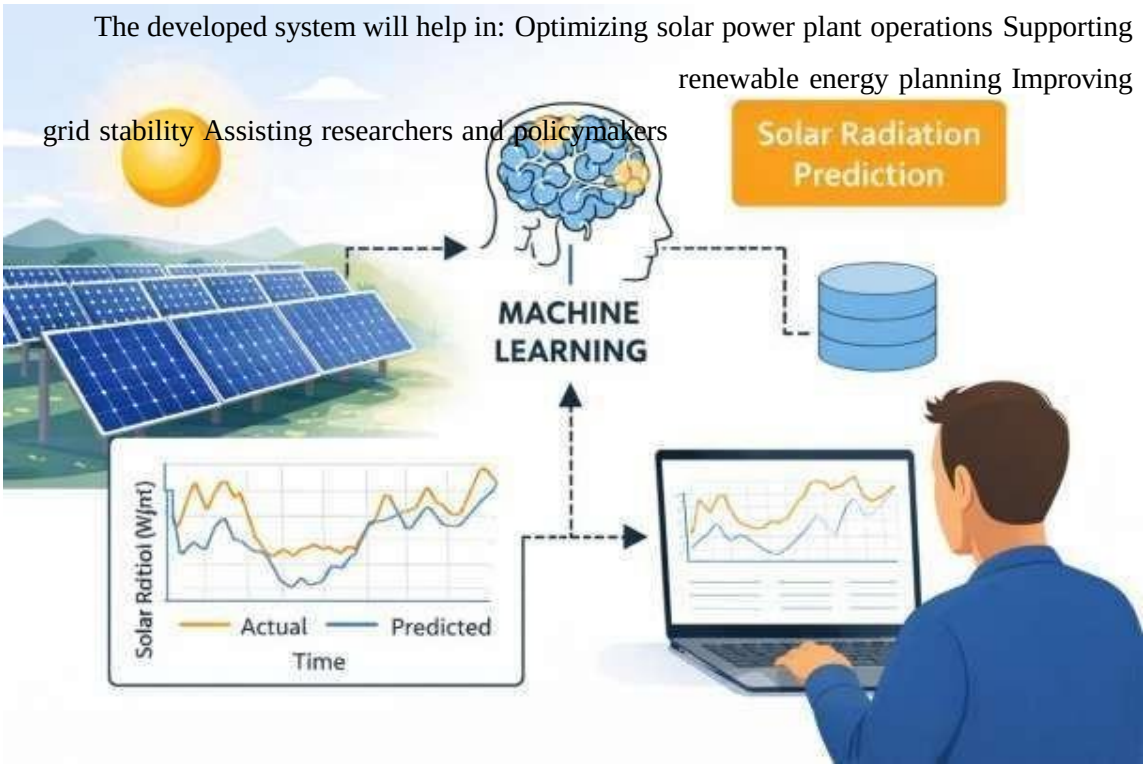



***Fig-1 :Introduction***

Thus, this project contributes toward sustainable energy development using modern Artificial Intelligence techniques.

## Objective

The primary objective of this project is to design and implement an intelligent Solar Radiation Prediction System using machine learning algorithms. The system aims to analyze historical meteorological data and generate accurate solar radiation forecasts.

The detailed objectives of the project are as follows:

### To Analyze Historical Solar Radiation Data

The first objective is to collect and study historical datasets containing solar radiation and weather-related parameters. Understanding the structure, trends, seasonal variations, and data distribution is essential before building prediction models.

### To Perform Data Preprocessing

Real-world datasets often contain missing values, duplicate records, noise, and outliers. Data preprocessing includes cleaning the data, handling missing values, normalizing numerical features, and encoding categorical variables. Proper preprocessing ensures better model performance.

### To Perform Feature Engineering

Feature engineering involves selecting the most relevant weather parameters that influence solar radiation. It may include transforming raw variables into meaningful features such as time-based attributes (hour, day, month), interaction features, and statistical measures.

### To Apply Machine Learning Algorithms

Various machine learning algorithms will be applied to the dataset, including:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor

Each model will be trained and tested separately to evaluate performance to Implement Ensemble Learning Techniques Ensemble learning combines multiple models to improve prediction accuracy. The objective is to integrate different algorithms using techniques such as: Voting Regressor, Bagging, Boosting

This helps in reducing bias and variance.

.

## To Evaluate Model Performance

The performance of models will be evaluated using metrics such as:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

Comparing these metrics helps identify the best-performing **$R^2$ Score** model.

### To Develop a User-Friendly Interface

The final objective is to build a simple and interactive interface where users can input weather parameters and receive predicted solar radiation values.

## Problem Statement

Solar radiation prediction is a challenging task due to the highly dynamic and nonlinear nature of atmospheric conditions. The amount of solar radiation reaching the Earth's surface changes continuously based on various environmental and meteorological factors. The key challenges in solar radiation prediction include:

### Changing Weather Conditions

Weather conditions such as temperature, humidity, and wind speed vary throughout the day. Sudden weather changes can significantly affect radiation levels.

### Cloud Cover Variations

Cloud cover is one of the most influential factors in determining solar radiation. Dense clouds can drastically reduce the radiation reaching solar panels.

### Seasonal Variations

Solar radiation levels differ across seasons. For example, summer months generally receive more radiation compared to winter months. Seasonal patterns must be captured accurately.

### Geographic Location Differences

Latitude, longitude, and altitude affect the angle and intensity of sunlight. Prediction models must consider location-based variations.

.

**Nonlinear Relationships**

The relationship between weather parameters and solar radiation is not linear. Traditional regression models often fail to model these complex relationships effectively.

- Data Quality Issues
- Real-world datasets may contain:
- Missing values
- Incomplete records
- Measurement errors
- Outliers

These issues reduce prediction accuracy if not handled properly.

Because of these challenges, traditional statistical methods are often insufficient for accurate solar radiation forecasting. There is a strong need for intelligent data-driven systems that can learn complex patterns from historical data and provide reliable predictions.

Therefore, this project proposes the use of advanced Machine Learning and Ensemble techniques to build a robust solar radiation prediction system. The system aims to improve accuracy, reduce forecasting errors, and support renewable energy management.

# CHAPTER-2
# LITERARURE REVIEW

# Introduction

Solar radiation forecasting has become an important research area in recent years due to the rapid growth of solar energy systems worldwide. As countries aim to reduce carbon emissions and shift toward renewable energy sources, solar power has emerged as one of the most promising alternatives to fossil fuels. However, solar energy production is highly dependent on weather conditions, which are uncertain and variable. This uncertainty creates challenges in power generation planning and grid stability.

To address these challenges, researchers have developed various forecasting models to predict solar radiation accurately. Early research mainly relied on physical and statistical models. Physical models used atmospheric physics and satellite observations to estimate radiation levels. Although these models provided scientific insights, they were often computationally complex and required detailed atmospheric data.

Later, statistical approaches such as regression models were introduced. These models attempted to establish mathematical relationships between solar radiation and meteorological parameters such as temperature, humidity, wind speed, and cloud cover. While statistical models were simpler to implement, they struggled to capture nonlinear and complex relationships present in real-world data. With the advancement of Artificial Intelligence and Machine Learning, researchers began applying ML algorithms to improve prediction accuracy. Machine learning models can learn patterns directly from data without relying on predefined equations. Over the past decade, numerous studies have demonstrated that ML- based models outperform traditional statistical techniques in solar radiation forecasting.

The literature in this field can be broadly categorized into:

- Statistical models
- Artificial Neural Network models
- Support Vector Machine models
- Tree-based models
- Ensemble learning models
- Deep learning models

This chapter reviews the major contributions made by researchers and highlights the strengths and limitations of different approaches.

## Review of Existing Research

### Statistical Models

In the early stages of solar radiation prediction research, Linear Regression was one of the most commonly used methods. Researchers attempted to model solar radiation as a linear combination of weather variables such as temperature, humidity, and sunshine duration.

**Advantages:**

- Simple to implement
- Easy to interpret
- Low computational cost Limitations:
- Assumes linear relationship
- Poor performance with nonlinear data
- Sensitive to outliers

Many studies concluded that while linear regression provides basic understanding, it is not suitable for highly complex atmospheric systems.

## Artificial Neural Networks (ANN)

Artificial Neural Networks were introduced to overcome the limitations of linear models. ANNs are inspired by the structure of the human brain and consist of interconnected neurons organized in layers. These networks are capable of modeling complex nonlinear relationships.

Several research studies reported significant improvements in prediction accuracy using ANN models. Multi-Layer Perceptron (MLP) networks were commonly applied for solar radiation forecasting.

**Advantages:**

- Captures nonlinear relationships
- High prediction accuracy
- Adaptive learning capability Limitations:
- Requires large datasets
- High computational cost
- Risk of overfitting
- Difficult to interpret

Although ANN models performed better than regression models, they required careful tuning of hyperparameters and longer training time.

## Support Vector Machines (SVM)

Support Vector Machines became popular due to their ability to perform well with smaller datasets. SVM works by finding an optimal hyperplane that best fits the data while minimizing error.

In solar radiation prediction, Support Vector Regression (SVR) was widely used. Research findings showed that SVR models provided stable performance and better generalization compared to ANN in some cases.

### Advantages:

- Effective for small and medium datasets
- Good generalization capability
- Less prone to overfitting Limitations:
- Selection of kernel function is complex
- Computationally expensive for large datasets
- Requires careful parameter tuning

Many researchers compared ANN and SVM models and observed that SVM sometimes provided better accuracy when data size was limited.

## Tree-Based Models

Decision Trees and Random Forest algorithms were later introduced to improve model stability and reduce overfitting.

### Decision Trees:

Decision Trees split the dataset based on feature values to make predictions. They are easy to interpret but prone to overfitting.

### Random Forest:

Random Forest is an ensemble of multiple decision trees. It combines predictions from different trees to produce a final output.

## Advantages

Handles nonlinear relationships Reduces overfitting Works well with large dataset Robust to noise Research studies demonstrated that Random Forest models significantly improved prediction accuracy compared to single decision trees and regression models.

## Ensemble Techniques

Ensemble learning combines multiple models to improve overall performance. Boosting and Bagging are two major ensemble techniques used in solar radiation forecasting.

### Gradient Boosting:

Gradient Boosting builds models sequentially, where each new model corrects the errors of the previous one. It has shown high accuracy in prediction tasks.

### XGBoost:

Extreme Gradient Boosting (XGBoost) is an advanced version of Gradient Boosting. It includes regularization to prevent overfitting and parallel processing for faster computation.

### Advantages:

- High prediction accuracy Handles missing data effectively Reduces bias and variance Efficient computation
- Recent research indicates that XGBoost and other boosting algorithms often outperform traditional ML models in solar radiation prediction tasks.

## Deep Learning Models

More recent studies have applied deep learning techniques such as: Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks are particularly useful for time-series forecasting because they can capture temporal dependencies.

### Advantages:

- Handles sequential data
- Captures long-term dependencies
- High accuracy for time-series data Limitations:
- Requires very large datasets
- High computational resources
- Complex model design

While deep learning models provide excellent results, they are more suitable for large-scale industrial applications.

### Methodology Used in Literature

Most research studies in solar radiation forecasting follow a structured methodology. The general workflow includes the following stages:

### Data Collection

Researchers collect historical solar radiation and meteorological data from:

- Weather stations
- Satellite observations
- Government meteorological departments
- Online datasets Common input features include:
- Temperature
- Humidity
- Wind speed
- Wind direction
- Atmospheric pressure
- Cloud cover
- Date and time

### Data Preprocessing

Raw data often contains inconsistencies. Preprocessing steps include:

- Handling missing values
- Removing duplicate entrie
- Outlier detection
- Normalization or standardization
- Encoding categorical variables
- Preprocessing improves data quality and enhances model performance.

### Feature Selection

Feature selection is used to identify the most important variables influencing solar radiation. Techniques include:

- Correlation analysis
- Principal Component Analysis (PCA)
- Recursive Feature Elimination (RFE)
- Feature importance from tree-based models
- Reducing irrelevant features improves accuracy and reduces computation time.

## Model Training

Different machine learning algorithms are trained using historical data. The dataset is usually split into:

- Training set (70–80%)
- Testing set (20–30%)
- Cross-validation is also used to ensure model reliability.

## Model Testing

I trained model is tested on unseen data to evaluate its performance. This step ensures the model generalizes well to new data. Performance Evaluation

Common evaluation metrics include:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- $R^2$ Score

Researchers compare multiple models and select the one with the best performance.


**Fig-2 :Model training**

Based on these findings, this project proposes the use of Ensemble Machine Learning techniques to achieve improved solar radiation prediction accuracy.

# CHAPTER-3
# SYSTEM ANALYSIS

# SYSTEM ANALYSIS

System analysis is an important phase in software development. It involves understanding the current system, identifying its limitations, and designing an improved solution. In this project, we analyze the existing solar radiation forecasting methods and propose a more efficient Machine Learning–based system.

Solar radiation prediction is complex because it depends on multiple environmental factors such as temperature, humidity, wind speed, cloud cover, atmospheric pressure, and seasonal changes. Traditional systems fail to handle these complex nonlinear relationships effectively. Therefore, a detailed system analysis is necessary to build a more reliable and intelligent solution.

Existing System

The existing solar radiation prediction systems mainly rely on:

- Statistical models
- Manual estimation methods
- Basic regression techniques
- Simple time-series forecasting models

**Statistical Models**

Traditional statistical models such as linear regression and time-series models (ARIMA) are commonly used. These models assume a linear relationship between input variables and output. However, solar radiation data is highly nonlinear and influenced by multiple interacting factors. As a result, these models often fail to capture complex patterns.

## Manual Estimation

In some small-scale solar installations, power output is estimated manually based on historical averages and seasonal trends. This approach lacks precision and does not consider real-time environmental variations.

**Basic Regression Models**

Simple regression models consider only a few features like temperature and time of day. They ignore other important weather parameters such as humidity, wind speed, and cloud cover. This results in incomplete modeling and inaccurate predictions.

## Disadvantages of Existing System

1. **Low Accuracy** Traditional statistical models fail to capture complex and nonlinear relationships in weather data, resulting in poor prediction performance.
2. **Inability to Handle Large Data** Conventional systems are not designed to efficiently process large volumes of historical weather datasets.
3. **Overfitting Issues:** Some regression-based models tend to overfit small datasets, which reduces their ability to perform well on new or unseen data.
4. **Poor Generalization** Existing systems often struggle to adapt to different geographical regions or seasonal variations.
5. **Limited Feature Utilization** Traditional approaches typically use fewer input parameters, limiting the overall prediction capability.
6. **Lack of Automation** Manual estimation methods require human intervention, making the process time-consuming and less efficient.
7. **Slow Decision-Making** Due to manual processes and limited computational capability, response time for predictions is slower.
8. **Scalability Issues** Expanding the system to include more parameters or larger datasets is difficult and inefficient.
9. **Limited Real-Time Prediction** Traditional systems are not well-suited for real-time weather forecasting.
10. **Reduced Adaptability** Updating the system with new data or changing environmental patterns is challenging.

These limitations highlight the need for an intelligent, scalable, and automated prediction system using modern machine learning techniques.

## Proposed System

The proposed system is a Machine Learning–based Solar Radiation Prediction system that uses advanced data-driven techniques. It focuses on improving prediction accuracy and reliability through ensemble learning.

The proposed system includes:

- Data preprocessing
- Feature engineering
- Ensemble ML algorithms
- Model comparison

- Graphical result visualization
- Data Preprocessing

Raw weather data often contains missing values, duplicate records, and inconsistent formats. The preprocessing module cleans and prepares the dataset by:

- Handling missing values
- Removing duplicate entries
- Normalizing data
- Encoding categorical variables
- Detecting and removing outliers
- Proper preprocessing improves model performance and ensures accurate predictions.
- Feature Engineering

Feature engineering plays a crucial role in prediction accuracy. Additional meaningful features such as:

- Hour of the day
- Day of the year
- Seasonal indicators
- Interaction features
- are created to enhance model learning capability.
- Ensemble Machi ne Learning Algorithms

Instead of relying on a single algorithm, the proposed system uses multiple ML models such as:

- Linear Regression
- Random Forest
- Gradient Boosting
- XGBoost

Finally, a Voting Regressor combines multiple models to produce more stable and accurate predictions.

### 3.2.1 Advantages of Proposed System

The proposed system offers several improvements compared to traditional approaches:

1. **Higher Accuracy** Ensemble machine learning models significantly improve prediction performance by combining multiple algorithms.
2. **Handles Nonlinear Data** Advanced machine learning algorithms can capture complex nonlinear relationships between weather parameters and solar radiation.
3. **Reduced Overfitting** Techniques such as Random Forest and Gradient Boosting minimize overfitting by combining multiple decision trees and optimizing performance.
4. **Automatic Prediction** After training, the system can automatically predict solar radiation for new input data without manual intervention.
5. **Scalable System** The system can efficiently handle large datasets and can be expanded by adding more features or data without major modifications

# CHAPTER-4
# FEASIBILITY STUDY

# 4. FEASIBILITY STUDY

Feasibility study is conducted to evaluate the practicality and viability of the proposed Fake Job Detection system. It determines whether the system can be successfully implemented from economic, technical, and social perspectives.

## 4.1 Economical Feasibility

The proposed Fake Job Detection system is economically feasible as it requires minimal financial investment for development and implementation. The project is developed using open-source technologies such as Python, Scikit-learn, Pandas, NumPy, and NLTK, which are freely available. This eliminates the need for expensive licensed software or paid development tools.

The system can operate on a standard computer with moderate specifications, and no high-end hardware or special infrastructure is required. This reduces the overall setup cost significantly.

Since the model can be saved and reused for future predictions, operational and maintenance costs are also low. Compared to manual fraud detection methods that require human resources, the automated Machine Learning approach reduces long-term expenses.

Therefore, the proposed system is cost-effective, practical, and suitable for academic as well as small to medium-scale industrial applications.

## 4.2 Technical Feasibility

The proposed Fake Job Detection system is technically feasible as it is built using well-established Machine Learning and Natural Language Processing techniques. The development tools such as Python, VS Code, and supporting libraries like Scikit-learn, Pandas, NumPy, and NLTK are widely available, easy to install, and well-documented.

The system uses TF-IDF for feature extraction and Random Forest for classification, both of which are reliable and commonly used algorithms for text classification problems. The dataset size is manageable, and the training process does not require high computational power or specialized hardware.

Additionally, the model can be saved and reused for predictions, making implementation and maintenance simple. Therefore, from a technical standpoint, the project is practical, efficient, and achievable with standard computing resources.

## 4.3 Social Feasibility

The proposed Fake Job Detection system is socially beneficial as it helps protect job seekers from fraudulent job postings. By automatically identifying fake advertisements, the system increases trust, transparency, and security in online recruitment platforms.

The system reduces the chances of financial fraud, identity theft, and misuse of personal information. It supports safer job searching by warning users about suspicious postings before they fall victim to scams.

Furthermore, the system does not negatively affect any social group. Instead, it contributes positively to society by promoting ethical recruitment practices and reducing online employment fraud. Therefore, from a social perspective, the project is acceptable, beneficial, and valuable for the community.


**Fig-3:Social Feasibility**

# CHAPTER-5
# SYSTEM REQUIREMENT

# SPECIFICATION

## 5. SYSTEM REQUIREMENT SPECIFICATION

The **System Requirement Specification (SRS)** defines the functional and non-functional requirements of the solar radio prediction system. It provides a comprehensive description of what the system is expected to perform, the features it must include, and the constraints under which it must operate. The SRS acts as a formal reference document that guides the design, development, testing, and deployment phases of the project.

This section clearly outlines the system's operational capabilities, including data processing, machine learning model implementation, prediction generation, and performance evaluation. It also specifies the technical resources, software tools, and hardware requirements necessary for successful implementation. By defining these requirements in advance, the SRS ensures that the development process remains structured, organized, and aligned with project objectives.

Additionally, the System Requirement Specification helps in minimizing ambiguity between user expectations and system functionality. It ensures that all stakeholders have a clear understanding of system behavior, performance standards, and quality attributes such as reliability, scalability, accuracy, and efficiency. Proper documentation of requirements also supports future enhancements, maintenance, and system upgrades.

Overall, the SRS serves as a foundational document that ensures the Fake Job Detection system meets user needs, operates efficiently within defined constraints, and delivers accurate and reliable results.

### 5.1 Functional Requirements

Functional requirements describe the specific operations and tasks that the system must perform. The system should allow users to input job posting details such as Temp, Clody, Area. The system must load and process the dataset containing solar radiation prediction

The system should perform text preprocessing operations including handling missing values, converting text to lowercase, removing punctuation and special characters, and eliminating stop words. It should combine relevant text fields to form a structured input for analysis.

The system must apply TF-IDF vectorization to transform textual data into numerical feature vectors suitable for machine learning algorithms.

The system should divide the dataset into training and testing sets for proper model evaluation. It must train a Random Forest classifier using the training dataset.

The system should evaluate the trained model using performance metrics such as Accuracy Score, Confusion Matrix, Classification Report (Precision, Recall, F1-Score), and Log Loss.

The system should accept new job posting input and predict whether it is Real or Fake. It must also display probability scores indicating the confidence level of the prediction.

The system should save the trained model and TF-IDF vectorizer using joblib so that they can be reused without retraining.

### 5.2 Non-Functional Requirements

Non-functional requirements define the quality standards and constraints under which the system operates.

The system should provide high classification accuracy and consistent performance. It should respond to user input within a short processing time.

The system should be user-friendly and easy to operate, even for users with basic technical knowledge. It must be reliable and stable during execution without frequent crashes or errors.

The system should be scalable, allowing future expansion to handle larger datasets or integration with online job portals. It should be maintainable and easy to update or modify if improvements are required.

The system should ensure data privacy and should not misuse or expose sensitive job-related information.

### 5.3 System Requirements

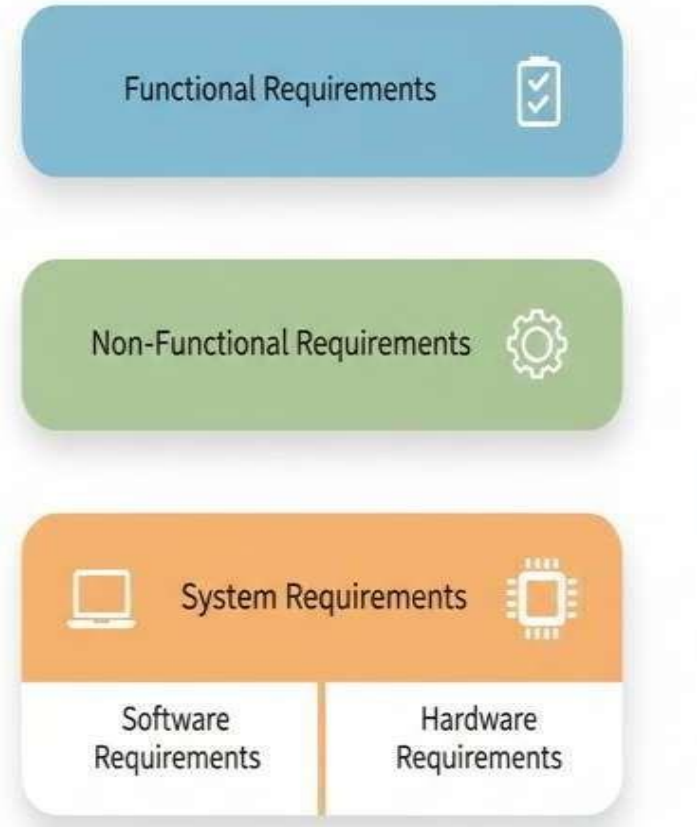



**Fig-:4 System requirements**

System requirements specify the technical environment needed to develop and execute the project successfully.

#### 5.3.1 Software Requirements

**Operating System:** Windows, Linux, or macOS

**Programming Language:** Python (3.x version recommended)

**Development Environment:** VS Code or Jupyter Notebook

**Libraries Used:** Pandas, NumPy, Scikit-learn, NLTK, Matplotlib, Joblib

**Optional Tools**: Git for version control

**Browser (if deployed as web application):** Google Chrome or Microsoft Edge

All required software tools are open-source and freely available, making installation and configuration simple.

### 5.3.2 Hardware Requirements

**Processor:** Intel i3 or higher (or equivalent)
**RAM:** Minimum 4 GB (8 GB recommended for better performance)
**Storage:** At least 10 GB of free disk space
**System Type:** 64-bit operating system

The hardware requirements for the solar radiation system are moderate and can be fulfilled by most modern personal computers or laptops. A standard system equipped with a basic processor such as Intel i3 or its equivalent and a minimum of 4 GB RAM is sufficient to execute the application effectively. For improved performance during data preprocessing and model training, 8 GB RAM is recommended, but it is not compulsory. The system does not rely on complex computations that demand high-end processing units.

Since the dataset used in this project is of manageable size, both training and prediction processes can be completed within a reasonable time using standard CPU resources. The Random Forest algorithm and TF-IDF vectorization are computationally efficient for medium-scale data and do not require parallel processing units or GPU acceleration. This ensures smooth performance even on entry-level systems.

The storage requirement is also minimal, as the dataset file and saved model files occupy limited disk space. A system with basic storage capacity (10 GB free space) is more than sufficient to handle the project files, required libraries, and generated outputs such as graphs and reports.

Furthermore, the absence of dependency on high-end servers or cloud infrastructure reduces operational complexity. The system can be developed, tested, and executed in a standalone environment without requiring network-based processing. This enhances portability and allows easy deployment in academic laboratories or personal systems.

Overall, the low hardware dependency makes the system cost-effective, accessible, and practical for students, educational institutions, startups, and small organizations. It ensures that the Fake Job Detection model can be implemented and maintained without significant infrastructure investment.

The System Requirement Specification clearly defines:

What the system must do (Functional Requirements)

How the system should perform (Non-Functional Requirements)

What resources are required (System Requirements)

The Solar Radiation Prediction System is designed to be efficient, scalable, secure, and user-friendly while delivering accurate predictions using machine learning techniques.


***Fig- :5 System Requirement Specification***

# CHAPTER-6
# SYSTEM DESIGN

# 6. SYSTEM DESIGN

System Design describes the overall structure and working architecture of the solar radiation prediction system. It explains how different components interact with each other to process input data and generate the final prediction. The design ensures that the system is modular, efficient, and easy to maintain.

## 6.1 System Architecture

The system architecture represents the structural framework of the solar radiation prediction. It consists of multiple interconnected modules that work together to perform fraud detection.

The architecture includes the following main components:

**1. User Input Layer**

**2. Preprocessing Layer**

**3. Feature Extraction Layer**

**4. Model Layer**

**5. Evaluation Layer**

**6. Output Layer**

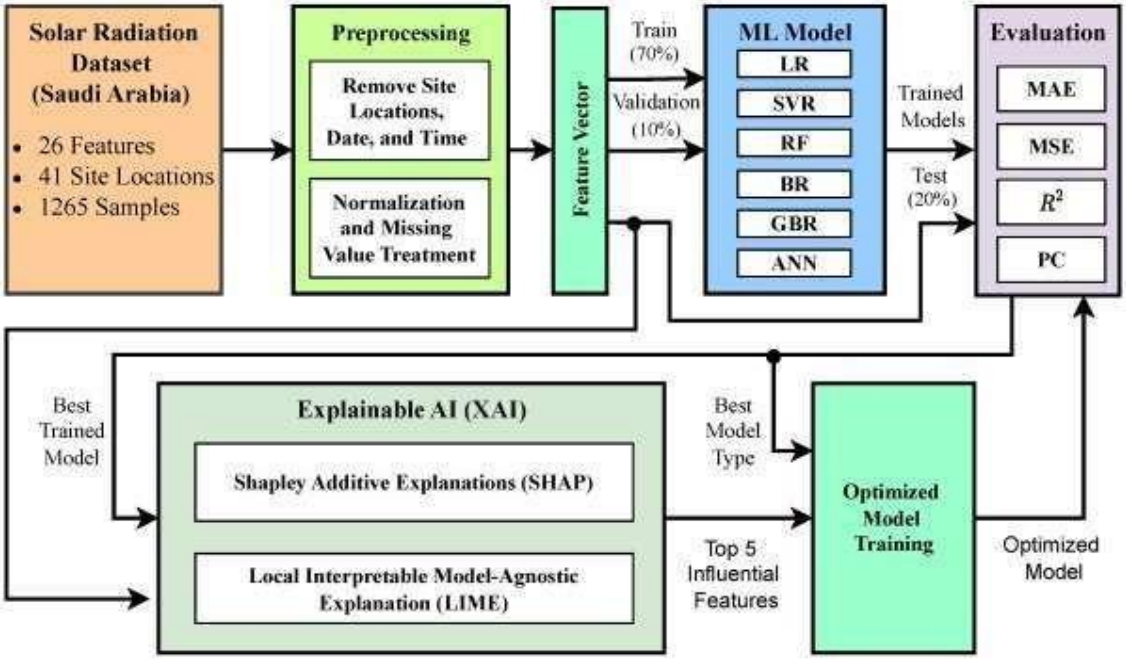



**Fig - 6**: **System Architecture**

## 6.2 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the solar radiation prediction system and how different processes interact with each other. It provides a visual representation of the flow of information from input to output, ensuring a clear understanding of system functionality.

The Data Flow Diagram (DFD) is a graphical representation of how data moves through the solar radiation prediction Detection system. It illustrates the flow of information between different processes, data stores, and external entities. The DFD helps in understanding the logical structure of the system and how each module interacts to produce the final output.

In this project, the DFD explains how job posting data is received, processed, transformed into numerical features, classified using the machine learning model, and finally displayed as a prediction result. It provides a clear view of how input data is systematically handled at each stage of the system.

The DFD is divided into different levels to provide both a high-level overview and a detailed internal process view:

The Level 0 DFD (Context Diagram) represents the solar radiation prediction system as a single process interacting with the external entity (User). It shows how the user provides job posting data as input and receives the classification result as output.

The Level 1 DFD breaks down the main system into multiple internal processes such as Data Input, Data Preprocessing, Feature Extraction, Model Classification, Evaluation, and Prediction Output. It shows how data flows between these processes and how each module contributes to generating the final result.

Overall, the Data Flow Diagram enhances the clarity of system design by visually explaining how data is processed from input to output within the Fake Job Detection system.

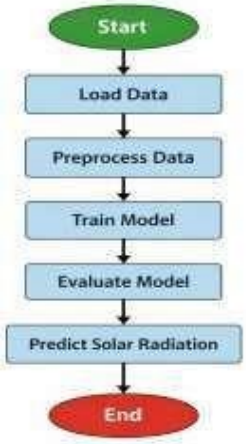



**Fig- 7:Data Flow Digram**

# CHAPTER-7
# IMPLEMENTATION

# 7. IMPLEMENTATION

Implementation is the phase where the designed system is transformed into a working application using appropriate tools and technologies. In this project, the solar radiation prediction system is implemented using Python along with various Machine Learning and Natural Language Processing libraries such as Scikit-learn, Pandas, NumPy, and Matplotlib. These tools provide the necessary functionalities for data handling, text processing, model building, and performance evaluation.

The implementation process begins with loading the dataset containing solar radiation prediction The textual data is then preprocessed to remove noise and irrelevant information. Feature extraction is performed using TF-IDF vectorization to convert text into numerical format suitable for machine learning algorithms. The processed data is split into training and testing sets, and a Random Forest classifier is trained to learn patterns that distinguish fraudulent postings from genuine ones.

After training, the model is evaluated using various performance metrics such as accuracy, confusion matrix, classification report, and log loss to measure its effectiveness. The trained model and vectorizer are saved for future use, enabling real-time predictions without retraining.

The system is developed in a modular manner, where each functionality is handled by a separate module. This modular approach ensures better clarity, easier debugging, improved maintainability, and flexibility for future enhancements. Overall, the implementation phase successfully converts the system design into a functional and efficient solar radiation prediction application.

## 7.1 Modules

The implementation of the Fake Job Detection system consists of the following modules:

The **Data Collection Module** is responsible for loading the dataset containing solar radiation prediction labeled as solar radiation. The dataset includes important fields such as Temp, Humidity, Pressure, Cloudy and the fraudulent label. This module ensures that the data is properly imported and structured for further processing.

The **Data Preprocessing Module** handles text cleaning operations to prepare the data for analysis. It performs tasks such as converting text to lowercase, removing punctuation and special characters, eliminating stopwords, and handling missing values. This module ensures that the textual data is clean, consistent, and meaningful before feature extraction.

The **Feature Extraction Module** applies TF-IDF vectorization to convert textual content into numerical feature vectors. Since machine learning models cannot process raw text directly, this module transforms the cleaned text into a structured numerical format suitable for classification.

The **Model Training Module** uses the Random Forest classifier to train the model on the training dataset. During this process, the model learns patterns and relationships within the data that help distinguish solar radiation prediction.

The **Model Evaluation Module** evaluates the trained model using performance metrics such as accuracy score, confusion matrix, classification report, and log loss. These metrics help measure the effectiveness and reliability of the classification model.

The **Model Saving Module** saves the trained model and TF-IDF vectorizer using joblib. This allows the system to reuse the trained components for future predictions without the need for retraining.

The **Prediction Module** accepts new job posting input and processes it through the trained system to classify it as solar radiation prediction. It also displays probability scores to indicate the confidence level of the prediction.

## 7.2 Source Code

The source code of the project is written in Python and organized into logical sections for better readability and maintenance. The implementation includes:

**Trained.ipynb**

[1]:
```
import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

[2]:
```
data = pd.read_csv(r"E:\solar_radiation_dataset.csv")

print("Dataset Shape:", data.shape)
```

```
Dataset Shape: (2000, 6)
```

[3]:
```
X = data.drop("Solar Radiation", axis=1)
y = data["Solar Radiation"]
```

[4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[5]:
```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', GradientBoostingRegressor())
])
```

[6]:
```
param_grid = {
    'model__n_estimators': [100, 200, 300],
    'model__learning_rate': [0.01, 0.05, 0.1],
    'model__max_depth': [3, 4, 5]
}

grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring='r2',
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

[7]:
```
best_model = grid.best_estimator_
```

[8]:
```
y_pred = best_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nModel Performance")
print("MAE :", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

[9]:
```
joblib.dump(best_model, "solar_best_model.pkl")

print("\nModel saved as solar_best_model.pkl")
```

**App.py**

```
from flask import Flask, render_template, request, jsonify
from flask import Flask
import joblib
import numpy as np

app = Flask(__name__)

# Load trained model
model = joblib.load("solar_best_model.pkl")

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["flOST"])
```

```
def predict():
    try:
        data = request.get_json()

        temperature = float(data["temperature"])
        humidity = float(data["humidity"])
        wind_speed = float(data["wind_speed"])
        pressure = float(data["pressure"])
        cloud_cover = float(data["cloud_cover"])

            input_data = np.array([[temperature, humidity,
wind_speed, pressure, cloud_cover]])

        prediction = model.predict(input_data)[0]

        return jsonify({
            "success": True,
            "prediction": round(float(prediction), 2)
        })

    except Exception as e:
        return jsonify({
            "success": False,
            "error": str(e)
        })

if __name__ == "__main__":
    app.run(debug=True)
```

**Index.html**

```
<!DOCTYflE  html>
<html lang="en">
<head>
    <meta  charset="UTF-8">
    <title>Solar  Radiation  flredictor</title>
    <link  rel="stylesheet"  href="../static/style.css">
</head>
<body>

<!-- NAVBAR -->
<nav class="navbar">
    <div  class="logo">☀ SolarAI</div>
    <ul  class="nav-links">
        <li><a  href="#home">Home</a></li>
        <li><a  href="#predict">flredict</a></li>
        <li><a  href="#sample">Sample  Values</a></li>
        <li><a  href="#dataset">Dataset  Info</a></li>
    </ul>
</nav>

<!-- HOME SECTION -->
<section  id="home"  class="section  home-section">
    <h1>🔶 Solar  Radiation  flredictor</h1>
    <p>AI flowered Weather Intelligence for Accurate Solar
Forecasting</p>
    <a  href="#predict"  class="primary-btn">Start  flrediction</a>
</section>

<!-- flREDICT  SECTION -->
<section  id="predict"  class="section">
    <div class="card">
        <h2>flredict  Solar  Radiation</h2>
```

```
            <div class="input-group">
                <label>Temperature (°C)</label>
                <input type="number" id="temperature">
            </div>

            <div class="input-group">
                <label>Humidity (%)</label>
                <input type="number" id="humidity">
            </div>

            <div class="input-group">
                <label>Wind Speed (m/s)</label>
                <input type="number" step="0.1" id="wind_speed">
            </div>

            <div class="input-group">
                <label>flressure (hfla)</label>
                <input type="number" id="pressure">
            </div>

            <div class="input-group">
                <label>Cloud Cover (%)</label>
                <input type="number" id="cloud_cover">
            </div>

            <button onclick="predictSolar()">flredict</button>

            <div class="result" id="resultBox">
                <h3 id="resultText"></h3>
            </div>
        </div>
    </section>
```

```
<!-- SAMflLE VALUES -->
<section id="sample" class="section light-section">
    <h2>Sample flredictions</h2>

    <div class="sample-grid">

        <div class="sample-card">
            <h3>Sample 1</h3>
            <p>Temp: 30°C</p>
            <p>Humidity: 45%</p>
            <p>Wind: 3.5 m/s</p>
            <p>flressure: 1012 hfla</p>
            <p>Cloud: 20%</p>
            <button class="output-btn high">
                High Radiation
                <span class="range">(800 - 1000 W/m²)</span>
            </button>
        </div>

        <div class="sample-card">
            <h3>Sample 2</h3>
            <p>Temp: 25°C</p>
            <p>Humidity: 60%</p>
            <p>Wind: 2.0 m/s</p>
            <p>flressure: 1008 hfla</p>
            <p>Cloud: 40%</p>
            <button class="output-btn medium">
                Medium Radiation
                <span class="range">(400 - 799 W/m²)</span>
            </button>
        </div>

        <div class="sample-card">
```

```
        <h3>Sample 3</h3>
        <p>Temp: 18°C</p>
        <p>Humidity: 75%</p>
        <p>Wind: 4.2 m/s</p>
        <p>flressure: 1005 hfla</p>
        <p>Cloud: 70%</p>
        <button class="output-btn low">
            Low Radiation
            <span class="range">(0 - 399 W/m²)</span>
        </button>
    </div>

    <div class="sample-card">
        <h3>Sample 4</h3>
        <p>Temp: 33°C</p>
        <p>Humidity: 35%</p>
        <p>Wind: 3.0 m/s</p>
        <p>flressure: 1015 hfla</p>
        <p>Cloud: 10%</p>
        <button class="output-btn high">
            High Radiation
            <span class="range">(800 - 1000 W/m²)</span>
        </button>
    </div>

    <div class="sample-card">
        <h3>Sample 5</h3>
        <p>Temp: 22°C</p>
        <p>Humidity: 65%</p>
        <p>Wind: 2.8 m/s</p>
        <p>flressure: 1009 hfla</p>
        <p>Cloud: 55%</p>
        <button class="output-btn medium">
```

```
                Medium Radiation
                <span class="range">(400 - 799 W/m²)</span>
            </button>
        </div>

        <div class="sample-card">
            <h3>Sample 6</h3>
            <p>Temp: 15°C</p>
            <p>Humidity: 80%</p>
            <p>Wind: 5.1 m/s</p>
            <p>flressure: 1003 hfla</p>
            <p>Cloud: 85%</p>
            <button class="output-btn low">
                Low Radiation
                <span class="range">(0 - 399 W/m²)</span>
            </button>
        </div>

        <div class="sample-card">
            <h3>Sample 7</h3>
            <p>Temp: 28°C</p>
            <p>Humidity: 50%</p>
            <p>Wind: 3.2 m/s</p>
            <p>flressure: 1011 hfla</p>
            <p>Cloud: 30%</p>
            <button class="output-btn high">
                High Radiation
                <span class="range">(800 - 1000 W/m²)</span>
            </button>
        </div>

    </div>
</section>
```

```
<!-- DATASET INFO --><section id="dataset" class="section">
    <h2> Dataset Overview</h2>

    <div class="dataset-grid">

        <div class="card">
            <h3> Features</h3>
            <ul>
                <li>Temperature</li>
                <li>Humidity</li>
                <li>Wind Speed</li>
                <li>flressure</li>
                <li>Cloud Cover</li>
            </ul>
        </div>

        <div class="card">
            <h3> Target Variable</h3>
            <p>Solar Radiation (W/m²)</p>
        </div>

        <div class="card">
            <h3> Data Size</h3>
            <p>Multiple Weather Observations</p>
            <p>Cleaned fi flreprocessed Dataset</p>
        </div>

        <!-- NEW DOWNLOAD CARD -->
        <div class="card">
            <h3> Dataset Access</h3>
            <p>Download full dataset for training fi
analysis.</p>
```

```
<a href="https://www.kaggle.com/datasets/karimipavan/solar-radiation-prediction" target="_blank">Kaggle Dataset</a>
        </div>

    </div>
</section>

<script src="{{ url_for('static', filename='script.js') }}"></script>
</body>
</html>
```

**Style.css**

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Segoe UI', sans-serif;
    scroll-behavior: smooth;
}
body {
        background: linear-gradient(135deg, #0f2027, #203a43, #2c5364);
    color: white;
}
.navbar {
    position: fixed;
    top: 0;
    width: 100%;
    padding: 15px 50px;
    display: flex;
    justify-content: space-between;
    align-items: center;
```

```
    background: rgba(255, 255, 255, 0.08);
    backdrop-filter: blur(12px);
    z-index:  1000;
}
.logo {
    font-size: 22px;
    font-weight: bold;
    letter-spacing: 1px;
}

.nav-links {
    list-style: none;
    display: flex;
    gap: 30px;
}

.nav-links a {
    text-decoration: none;
    color: white;
    font-weight: 500;
    transition: 0.3s ease;
}

.nav-links a:hover {
    color:  #00f2fe;
}

.section {
    padding: 120px 40px;
    min-height: 100vh;
    text-align: center;
}
```

```
.light-section {
    background:  rgba(255,255,255,0.05);
}
.home-section {
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    gap: 15px;
}

.home-section h1 {
    font-size: 42px;
}

.primary-btn {
    margin-top: 20px;
    padding:  12px  30px;
    background: linear-gradient(45deg, #00f2fe, #4facfe);
    border-radius: 30px;
    text-decoration: none;
    color: white;
    font-weight: bold;
    transition: 0.3s ease;
}

.primary-btn:hover {
    transform: scale(1.08);
}

#predict {
    display: flex;
    justify-content:  center;
```

```
    align-items: center;
}
#predict .card {
    width: 420px;
    padding: 40px;
    border-radius: 25px;
     background: linear-gradient(145deg, rgba(255,255,255,0.08),
rgba(0,0,0,0.3));
    border: 1px solid rgba(255,255,255,0.15);
    backdrop-filter: blur(15px);
    box-shadow: 0 0 40px rgba(0, 255, 200, 0.1);
    color: white;
    transition: 0.4s ease;
}

#predict .card:hover {
    box-shadow: 0 0 50px rgba(0, 255, 200, 0.25);
}

/* Heading */
#predict h2 {
    margin-bottom: 25px;
    color: #00ffcc;
}

/* Input Groups */
#predict .input-group {
    margin-bottom: 18px;
    text-align: left;
}

#predict label {
    font-size: 14px;
```

```
    color:  #ddd;
}

#predict  input  {
    width: 100%;
    padding: 10px 12px;
    margin-top: 6px;
    border-radius:  10px;
    border: 1px solid rgba(255,255,255,0.2);
    background: rgba(255,255,255,0.08);
    color: white;
    outline: none;
    transition: 0.3s ease;
}

#predict input:focus {
    border: 1px solid #00ffcc;
    box-shadow: 0 0 10px #00ffcc;
}

/* flredict Button */
#predict button {
    margin-top: 15px;
    padding: 12px;
    border-radius: 12px;
    border: none;
    background: linear-gradient(45deg, #00ffcc, #00c6ff);
    color: black;
    font-weight: bold;
    font-size: 16px;
    cursor: pointer;
    transition: 0.3s ease;
}
```

```
#predict button:hover {
    transform: scale(1.05);
    box-shadow: 0 0 20px #00ffcc;
}

/* Result Box */
#predict .result {
    margin-top: 20px;
    padding: 15px;
    border-radius: 12px;
    background: rgba(0, 255, 179, 0.1);
    border: 1px solid rgba(0,255,179,0.3);
    color: #00ffcc;
    font-weight: 600;
    display: none;
    animation: fadeIn 0.4s ease-in-out;
}

.sample-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 30px;
    margin-top: 50px;
}

.sample-card {
    background: rgba(255,255,255,0.08);
    padding: 25px;
    border-radius: 20px;
    backdrop-filter: blur(12px);
    text-align: left;
    transition: 0.4s ease;
```

```
}

.sample-card:hover {
    transform:  translateY(-10px);
    box-shadow: 0 20px 40px rgba(0,0,0,0.4);
}

.sample-card p {
    font-size: 14px;
    color: #f1f1f1;
    margin: 6px 0;
    padding: 6px 10px;
    background: rgba(255, 255, 255, 0.05);
    border-radius: 8px;
    display: flex;
    justify-content: space-between;
    transition: 0.3s ease;
}

.sample-card p:hover {
    background: rgba(255, 255, 255, 0.12);
    transform: translateX(4px);
}

.sample-card p b {
    color: #00f2fe;
}

/* OUTPUT BUTTONS */

.output-btn {
    margin-top: 15px;
    padding: 8px 18px;
```

```
    border: none;
    border-radius: 20px;
    font-weight: bold;
    cursor: default;
}

.output-btn.high {
    background: #28a745;
    color: white;
    box-shadow: 0 0 18px #28a745;
}

.output-btn.medium {
    background: #ffc107;
    color: black;
    box-shadow: 0 0 18px #ffc107;
}

.output-btn.low {
    background: #dc3545;
    color: white;
    box-shadow: 0 0 18px #dc3545;
}

/* ================ DATASET SECTION ================ */

#dataset h2 {
    color: #00ffb3;
    margin-bottom: 40px;
    font-size: 32px;
}

/* Dataset Grid */
```

```
.dataset-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 30px;
    margin-top:  20px;
}

/* Neon Glass Dataset Cards */
#dataset .card {
    border: 1px solid rgba(255,255,255,0.2);
     background: linear-gradient(145deg, rgba(255,255,255,0.05),
rgba(0,0,0,0.25));
    box-shadow: 0 0 25px rgba(0, 0, 0, 0.5);
    padding: 30px;
    border-radius: 20px;
    backdrop-filter: blur(12px);
    transition: 0.4s ease;
    text-align: left;
    color: #eee;
}

#dataset .card:hover {
    transform: translateY(-8px);
    box-shadow: 0 0 35px rgba(0, 255, 179, 0.3);
}

#dataset .card h3 {
    margin-bottom: 15px;
    color: #00ffcc;
}

#dataset ul {
    list-style:  none;
```

```
    padding-left:  0;
}

#dataset ul li {
    padding: 10px 0;
    border-bottom:  1px solid rgba(255,255,255,0.1);
}

#dataset ul li:last-child {
    border-bottom: none;
}

#dataset p {
    padding: 8px 0;
}

/* Dataset Button */
#dataset a {
    margin-top: 15px;
    color: #00ffcc;
    text-decoration: none;
    font-weight: 600;
    padding: 10px 20px;
    border-radius: 10px;
    background: rgba(0, 255, 204, 0.15);
    border: 1px solid #00ffcc;
    transition: 0.3s ease;
    display:  inline-block;
}

#dataset a:hover {
    background: #00ffcc;
    color: black;
```

```
    box-shadow: 0 0 20px #00ffcc;
    transform: scale(1.05);
}

/* ================ RESflONSIVE ================ */

@media(max-width: 768px) {

    .navbar {
        flex-direction: column;
        gap: 10px;
    }

    .nav-links {
        flex-direction: column;
        gap: 15px;
    }

    .card {
        width: 100%;
    }

    .section {
        padding: 120px 20px;
    }
}
```

## Script.js

```
async function predictSolar() {
    const temperature =
document.getElementById("temperature").value;
    const humidity = document.getElementById("humidity").value;
    const wind_speed =
document.getElementById("wind_speed").value;
    const pressure = document.getElementById("pressure").value;
    const cloud_cover =
document.getElementById("cloud_cover").value;

    const resultBox = document.getElementById("resultBox");
    const resultText = document.getElementById("resultText");

    if (!temperature || !humidity || !wind_speed || !pressure ||
!cloud_cover) {
        alert("fllease fill all fields!");
        return;
    }

    const response = await fetch("/predict", {
        method: "flOST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            temperature,
            humidity,
            wind_speed,
            pressure,
            cloud_cover
        })
    });
```

```
        const data = await response.json();

        resultBox.style.display = "block";

        if (data.success) {
            resultBox.className = "result success";
            resultText.innerText = "● flredicted Solar Radiation: "
+ data.prediction;
        } else {
            resultBox.className = "result error";
            resultText.innerText = "Error: " + data.error;
        }
}
```

# CHAPTER-8
# SOFTWARE ENVIRONMENT

# Software Environment

The successful implementation of the Student Performance Prediction System requires an appropriate software environment that supports data analysis and machine learning operations.

## Operating System

The system was developed and executed on a Windows operating system. Windows provides a stable platform for installing Python and related data science libraries.

## Programming Language

Python 3.10 or above was used as the primary programming language for this project. Python is the industry standard for machine learning due to its simplicity, readability, and extensive support for scientific libraries.

### 8.1 Backend Framework: Flask

Flask is a lightweight and flexible web framework for Python used to develop web applications and APIs. In this project, Flask is used as the backend framework to handle communication between the frontend interface and the trained deep learning model.

Flask plays an important role in integrating the Convolutional Neural Network (CNN) model with the web application. It manages image uploads, processes requests, loads the trained .h5 model, and returns prediction results to the user interface.

Flask is used in this project to:

- Handle HTTP requests from the frontend
- Receive uploaded Brain tumor detection
- Perform image validation and preprocessing
- Load the trained CNN model (.h5 file)
- Generate disease prediction results
- Send prediction results back to the frontend

## Advantages of Flask:

- Lightweight and easy to implement
- Easy integration with TensorFlow and Keras
- Flexible and scalable architecture
- Minimal configuration required

- Efficient for machine learning-based web applications

Flask enables smooth communication between the frontend and backend and ensures real- time plant disease detection.

## 8.2 Frontend Technology: HTML, CSS, JavaScript

The frontend of the system is developed using HTML, CSS, and JavaScript. The frontend provides the user interface through which users can upload MRI images and view
prediction results.
HTML is used to structure the web page, CSS is used to design and style the interface, and JavaScript is used to enhance interactivity and handle form submissions.

The frontend is responsible for:

- Displaying image upload form
- Showing uploaded image preview
- Sending image data to backend
- Displaying predicted disease result
- Showing error messages if necessary

## Advantages of HTML, CSS, JavaScript:

- Simple and widely supported technologies
- Responsive and user-friendly interface
- Easy integration with Flask backend
- Fast loading and efficient performance

The frontend communicates with Flask backend using HTTP requests to display real-time prediction results.

## 8.3 Model Storage: .h5 File (TensorFlow/Keras Model)

In this project, the trained Convolutional Neural Network model is saved in .h5 format. The .h5 file stores the model architecture, trained weights, and optimizer configuration. The .h5 model file is used to:

- Load trained CNN model during runtime
- Perform real-time disease prediction
- Avoid retraining the model repeatedly

- Improve system efficiency

## Advantages of using .h5 format:

- Stores complete model information
- Easy to load and deploy
- Reduces computational cost
- Suitable for production deployment

The backend loads the .h5 file when the server starts, ensuring fast and efficient predictions.

## 8.4 Jupyter Notebook

In this project, Anaconda and Jupyter Notebook were used as the primary development environment for building and training the Convolutional Neural Network (CNN) model for plant disease detection.

Anaconda is a Python distribution platform that simplifies package management and environment configuration. It provides a stable environment for installing required libraries such as TensorFlow, Keras, NumPy, Matplotlib, and OpenCV. In this project, Anaconda was used to create a dedicated virtual environment to avoid dependency conflicts and ensure smooth execution of machine learning libraries. The Anaconda Navigator interface was used to manage environments and launch development tools easily.

Jupyter Notebook, launched through Anaconda, was used for developing and training the deep learning model. It provides an interactive, cell-based coding environment that allows step-by-step execution of Python code. This was particularly useful for preprocessing the Brain Tumor dataset, designing the CNN architecture, training the model, and evaluating performance. Accuracy and loss graphs were plotted within the notebook to analyze the model's performance before saving it in .h5 format.

The workflow followed in this project was:

- Install required libraries using Anaconda
- Create a dedicated Python environment
- Launch Jupyter Notebook from Anaconda Navigator
- Load and preprocess the dataset
- Design and train the CNN model
- Evaluate model performance

- Save the trained model in .h5 format

The combination of Anaconda and Jupyter Notebook provided a structured, efficient, and organized development environment for machine learning experimentation and modelntraining.

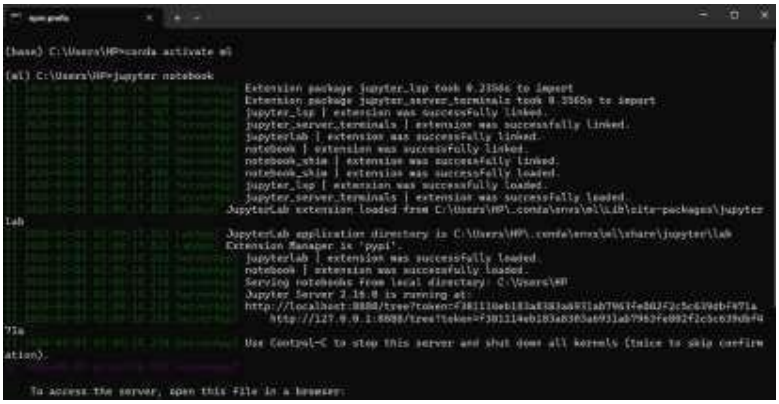



**Fig-8 : Anaconda Prompt**

**Fig-9: Jupyter notebook**

## 8.5 Visual Studio

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. In this project, VS Code was used as the primary development tool for building the web application component of the Deep Learning-Based Plant Disease Identification System. While Jupyter Notebook was used for model training, VS Code was used for integrating the trained CNN model with the Flask backend and developing the frontend interface.

VS Code provides a user-friendly interface with advanced features such as syntax highlighting, IntelliSense (auto-completion), debugging tools, and integrated terminal support. These features made the development process more efficient and organized. The Python extension in VS

Code was installed to support Python programming and Flask development.

The project directory was structured and managed using VS Code. Separate folders were maintained for templates (HTML files), static files (CSS and images), and model files (.h5). This organized structure improved maintainability and readability of the project.

VS Code also provides an integrated terminal, which was used to run Flask commands such as starting the development server. This allowed real-time testing of the application in a local browser environment.

The use of VS Code provided several advantages:

- Lightweight and fast performance
- Easy integration with Python and Flask
- Built-in debugging support
- Organized project management

The workflow in this project involved first training the CNN model in Jupyter Notebook and saving it as a .h5 file. The saved model was then imported into the Flask application developed in VS Code. The backend code was written to load the model, preprocess images, and generate predictions. The frontend interface was connected to the backend using Flask routes. Overall, Visual Studio Code played a critical role in converting the trained machine learning model into a functional web application. It provided a structured and efficient development environment for integrating artificial intelligence with web technologies

# CHAPTER-9
# SYSTEM TESTING

## 9.1 Testing Strategies

Testing strategies define the systematic approach used to verify that the Deep Learning-Based Plant Disease Identification System functions correctly and meets all specified requirements. Since this project integrates Artificial Intelligence, image processing, and web technologies, a structured multi-level testing approach was adopted to ensure reliability, accuracy, and performance.

The testing process focused on validating both software functionality and machine learning model behavior. Different testing strategies were implemented to evaluate individual components, integration flow, system performance, and prediction accuracy.

### Unit Testing

Unit testing was performed to verify individual modules independently. Each module, such as image upload, preprocessing, model loading, and prediction, was tested separately to ensure correct functionality.

For example, the preprocessing module was tested to confirm proper resizing and normalization of input images. The model loading process was verified to ensure the .h5 file loads successfully without runtime errors. This strategy helped identify errors at an early stage and improved module reliability before integration.

### Integration Testing

Integration testing was conducted to verify smooth communication between system components. Since the project combines frontend technologies (HTML, CSS, JavaScript), backend framework (Flask), and a deep learning model (TensorFlow/Keras), it was essential to test data flow across layers.

The following interactions were tested:

- Image upload from frontend to Flask backend
- Pre-processed image transfer to CNN model
- Model prediction returned to backend
- Prediction result displayed on frontend

This ensured that all components work together seamlessly.

## Functional Testing

Functional testing was carried out to verify that the system satisfies all functional requirements. The application was tested with multiple plant leaf images, including healthy and diseased samples, to confirm correct classification.

The system was validated for:

- Accurate disease prediction
- Proper display of prediction results
- Handling invalid file formats
- Supporting repeated image uploads

This testing ensured that the application behaves according to its intended design.

## Performance Testing

Performance testing was performed to evaluate system efficiency and response time. Since the application is designed for real-time disease detection, prediction results were expected within a few seconds. The backend server performance and model inference time were monitored to ensure:

- Fast image processing
- Stable server performance
- Minimal response delay

The system consistently generated results within acceptable time limits.

## Accuracy Testing

Accuracy testing focused on evaluating the effectiveness of the trained CNN model. The model was tested using unseen test dataset images to measure classification accuracy. Training accuracy and validation accuracy metrics were analyzed to ensure reliable performance.

This strategy confirmed that the model generalizes well and produces consistent predictions for different Brain MRI Images .

# CHAPTER-10
# SCREENSHOTS

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | Temperature | Humidity | Wind Speed | Pressure | Cloud Cover | Solar Radiation |
| 2 | 26 | 52 | 1.187855452 | 1004 | 11 | 300.004761 |
| 3 | 39 | 59 | 1.639334006 | 1001 | 9 | 589.429745 |
| 4 | 34 | 37 | 4.827733753 | 1008 | 70 | 229.1534756 |
| 5 | 30 | 51 | 1.000058174 | 1003 | 54 | 100 |
| 6 | 27 | 73 | 3.082829142 | 1016 | 0 | 288.6456012 |
| 7 | 26 | 65 | 3.612550905 | 1016 | 48 | 100 |
| 8 | 38 | 34 | 1.2731721 | 1017 | 35 | 535.396889 |
| 9 | 30 | 42 | 5.865390815 | 1015 | 20 | 424.190083 |
| 10 | 30 | 58 | 2.130626656 | 1007 | 36 | 171.6160765 |
| 11 | 23 | 60 | 2.520993598 | 1010 | 23 | 100 |
| 12 | 27 | 76 | 2.519712561 | 1017 | 54 | 100 |
| 13 | 22 | 45 | 2.152083311 | 1009 | 24 | 154.9274637 |
| 14 | 21 | 71 | 1.00736911 | 1012 | 6 | 100 |
| 15 | 31 | 59 | 4.64672397 | 1016 | 80 | 100 |
| 16 | 25 | 45 | 5.834227499 | 1017 | 92 | 100 |
| 17 | 21 | 41 | 2.121467416 | 1009 | 93 | 100 |
| 18 | 20 | 46 | 4.315235959 | 1015 | 70 | 100 |
| 19 | 31 | 45 | 4.709481635 | 1019 | 96 | 100 |
| 20 | 31 | 41 | 5.242126895 | 1004 | 70 | 100 |

**Fig- 10: Dataset Image**


**Fig- : Home Page**


***Fig- 11: Predict Page***

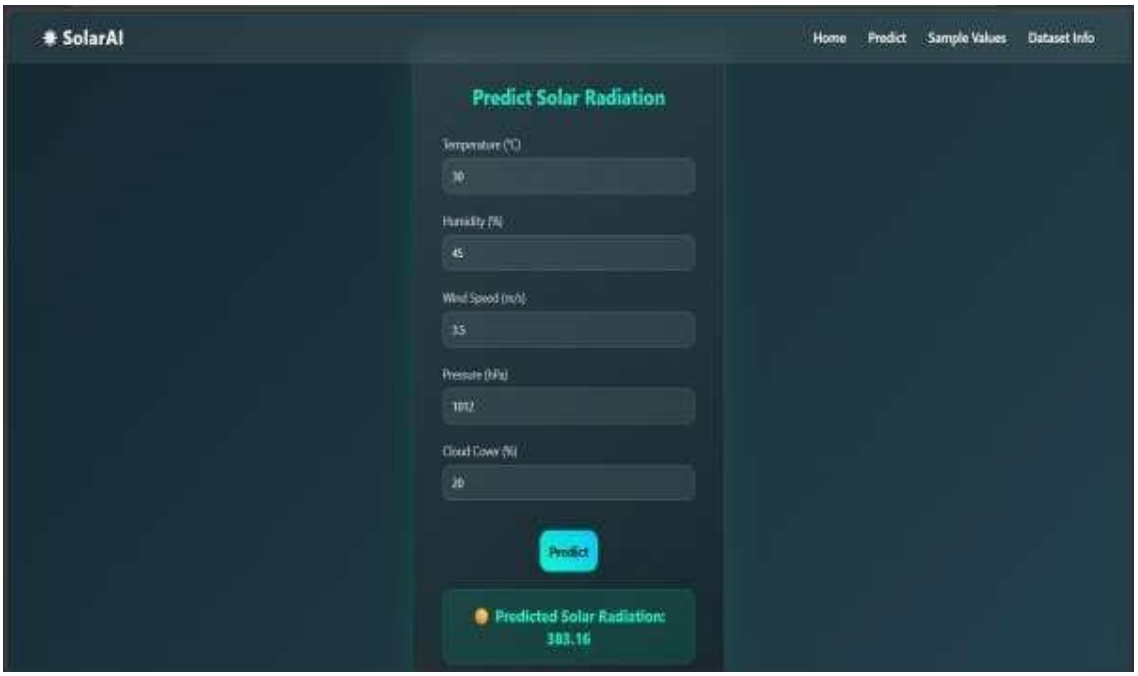



**Fig- 12: Predict and Result Page**


**Fig- 13: Sample Inputs and Outputs**

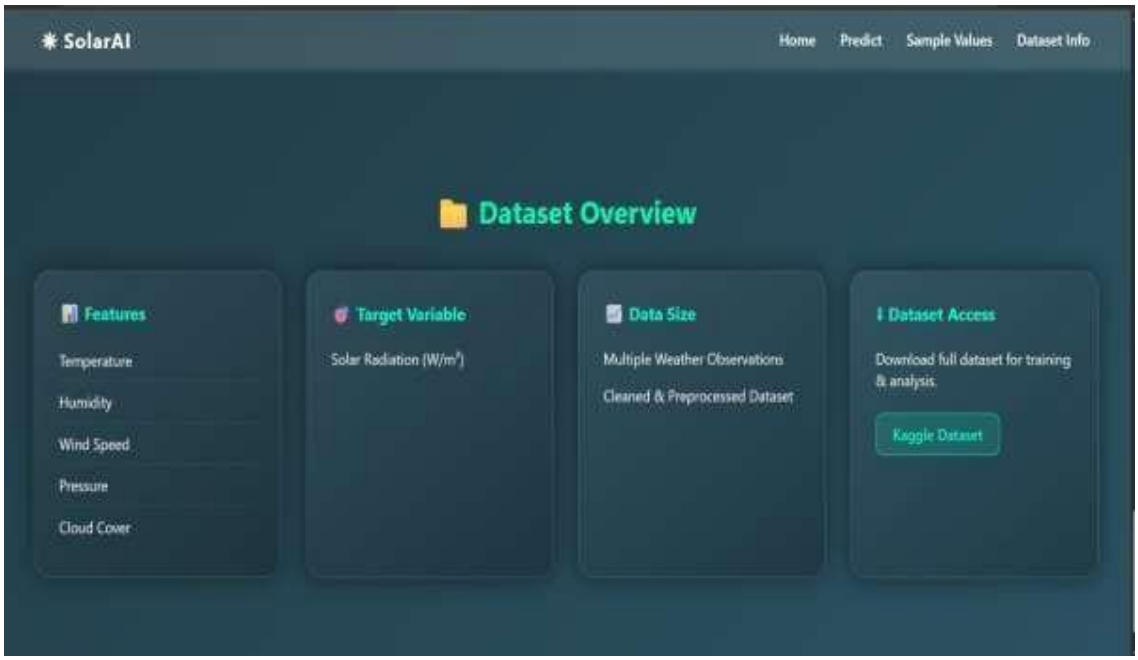



**Fig-14 : Dataset Info**

# CHAPTER-11
# CONCLUSION

# CONCLUSION

The **Solar Radiation Prediction using Machine Learning** project demonstrates how data-driven approaches can significantly enhance renewable energy forecasting. Accurate solar radiation prediction is essential for efficient solar power plant operation, grid stability, and smart energy management. By improving forecasting accuracy, the system helps reduce energy wastage, optimize power generation planning, and lower operational costs.

In this project, multiple machine learning algorithms were implemented and evaluated, including Linear Regression, Random Forest, Gradient Boosting, and XGBoost. Linear Regression served as a baseline model but showed limitations in handling nonlinear patterns. Random Forest improved prediction performance by reducing overfitting through multiple decision trees. Gradient Boosting and XGBoost further enhanced accuracy by sequentially correcting prediction errors and capturing complex relationships in weather data.

The most significant improvement was achieved using an ensemble approach through a Voting Regressor that combined Random Forest, Gradient Boosting, and XGBoost. Ensemble learning proved highly effective, as combining multiple models reduced bias and variance while increasing overall prediction stability and reliability. Performance evaluation was carried out using standard metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and $R^2$ Score, confirming the superiority of the ensemble model over individual algorithms.

The project successfully handled data preprocessing tasks, including missing value treatment, normalization, and feature engineering. It also demonstrated that open-source tools like Python and Scikit-learn are sufficient to develop practical and scalable forecasting systems.

From an application perspective, accurate solar radiation forecasting supports better power generation planning, enhances grid stability, reduces dependence on backup power systems, and promotes environmental sustainability. Such systems are particularly valuable for smart grids and future smart city infrastructures.

However, certain limitations remain. Prediction accuracy depends heavily on dataset quality and size. Extreme weather conditions may impact reliability, and real-time data integration was not implemented. Additionally, deep learning models were not explored in this version.

Overall, the project successfully builds a reliable and scalable solar radiation prediction system using machine learning techniques. With future enhancements such as deep learning integration, real-time data streaming, and cloud deployment, the system can evolve into a large-scale industrial solution. This work highlights the important role of artificial intelligence in renewable energy management and contributes toward building a sustainable and energy-efficient future.

# CHAPTER-12
# FEATURE ENHANCEMENT

Future improvements can enhance prediction accuracy by integrating a wider range of meteorological variables and adopting advanced modeling techniques. In addition to basic weather parameters, factors such as aerosol concentration, precipitation, solar zenith angle, and atmospheric turbidity can be incorporated to better capture environmental complexity. Advanced deep learning models like LSTM, GRU, CNN-LSTM, and transformer-based architectures can further improve time-series forecasting by learning long-term dependencies and spatial–temporal patterns. Automated hyperparameter optimization techniques such as Grid Search, Bayesian Optimization, and AutoML can also be applied to systematically identify optimal model configurations and maximize performance.

The system can be strengthened by integrating satellite imagery and real-time meteorological data from APIs or IoT sensors to enable dynamic and location-specific forecasting. Expanding geographical generalization through transfer learning will allow the model to adapt to different climatic regions with minimal additional training. Deployment as a cloud-based web or mobile application can improve accessibility and scalability, providing real-time dashboards and downloadable reports. Furthermore, integrating solar radiation prediction with photovoltaic power output estimation and extending the model for long-term forecasting will support smart grid management, renewable energy planning, and sustainable infrastructure development.